

Party Monopoly

Neon Night Market

Technical Design Document

A deterministic Monopoly engine whose key moments break into minigames

reflex over luck

Contents

1	Overview	2
1.1	Design pillars	2
1.2	The core loop	2
1.3	Build phases	2
2	Architecture	2
2.1	Source-as-entry workspaces	3
2.2	TypeScript project setup	3
3	The Engine	3
3.1	Determinism	3
3.2	Game state	4
3.3	Turn phases	4
3.4	Actions and events	4
3.5	Square resolution and the rent flow	5
3.6	Win conditions	5
4	The Minigame Harness	5
4.1	Interface and registry	5
4.2	Reflex Tap Duel	5
4.3	Adjudication	6
5	The AI	6
5.1	Turn decisions	6
5.2	Duel reflexes	6
6	The Online Server	6
6.1	Room lifecycle	7
6.2	Action handling and the legality gate	7
6.3	Server-run showdown	7
6.4	Wire protocol	7
7	The Web Client	7
7.1	Modes	7
7.2	Components	8
7.3	Stores and telemetry	8
8	Economy and Tuning	8
8.1	Default tunables	8
8.2	How rent is priced	9
8.3	The convergence problem	9
8.4	The interim fix (shipped)	9
8.5	Measuring convergence — the sim harness	9
8.6	Open question — elimination vs. points	9
9	Building, Testing, Running	10
9.1	Scripts	10
9.2	Tests	10
9.3	Reproducing a game	10

10 Glossary

11

1 Overview

Party Monopoly is a Monopoly variant built around one inversion of the original game: *the moments that normally resolve with arithmetic instead resolve with a minigame*. The first version proves a single instance of this idea, the **Rent Showdown** — when a player lands on an opponent's property, rent is not simply paid; the two players fight a brief two-player reflex duel, and the result scales what the rent is worth.

This document is the technical reference for the project: the architecture, the deterministic engine, the minigame seam, the AI, the authoritative online server, the React client, and the economy model with its current open questions. It is written to be read by both designers and engineers.

1.1 Design pillars

- **Reflex over luck.** The headline mechanic rewards a skill the dice never test. The engine stays a faithful Monopoly skeleton so the twist reads as a twist.
- **Deterministic core.** A game is fully reproducible from its `seed` plus its action log. No wall clock and no `Math.random` ever touch the engine.
- **One mechanic, proven well.** The MVP ships exactly one minigame (Reflex Tap Duel) wired into exactly one trigger (rent). Everything else is a deliberately thin, faithful board.
- **Transport independence.** The same reducer drives hotseat, single-player-vs-AI, and authoritative online play. The transport changes; the rules do not.

1.2 The core loop

1. Active player rolls; doubles grant another roll (three in a row sends them to jail).
2. The pawn moves; passing GO pays a salary.
3. The landed square resolves by type. Landing on an *unowned* property offers a buy decision; landing on an *opponent-owned* property opens a Rent Showdown.
4. The duel decides an outcome (payer wins / owner wins / draw), which selects a rent multiplier. Rent is transferred; an insolvent payer goes bankrupt to the owner.
5. Turn ends, the pointer advances, laps are counted, and win conditions are checked.

1.3 Build phases

1. **Engine playable in hotseat** — the deterministic reducer and a local client. (*complete*)
2. **Reflex Tap Duel wired into Rent Showdown** — the minigame seam and its renderer. (*complete*)
3. **Online play** — the engine wrapped in a Colyseus room with the server adjudicating duels. (*complete*)

2 Architecture

The repository is an npm-workspaces monorepo. Six packages form a strict dependency layering: **types** depends on nothing, and everything depends on **types**. The engine and the minigame harness sit on top of types; the AI, server, and web client sit on top of those.

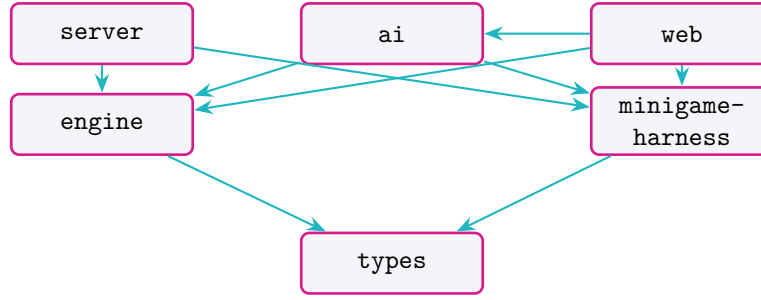


Figure 1: Package dependency graph. Arrows point from dependent to dependency.

Package	Role	Depends on
<code>types</code>	Branded ids, action/event/minigame/wire types	—
<code>engine</code>	Pure deterministic reducer, board, RNG, tunables	<code>types</code>
<code>minigame-harness</code>	Minigame interface, registry, reflex adjudication	<code>types</code>
<code>ai</code>	Deterministic bot decisions and duel reflexes	<code>engine</code> , <code>harness</code>
<code>server</code>	Authoritative Colyseus room wrapping the engine	<code>engine</code> , <code>harness</code>
<code>sim</code>	Headless bot-vs-bot balance simulation	<code>engine</code> , <code>harness</code> , <code>ai</code>
<code>apps/web</code>	React + Vite client, Zustand state mirror	<code>engine</code> , <code>harness</code> , <code>ai</code>

Table 1: Packages and their dependencies.

2.1 Source-as-entry workspaces

Each package's `package.json` points `main` and `types` directly at `./src/index.ts`. Consumers resolve TypeScript source straight from the workspace, so no per-package build step is needed during development. Workspace dependencies are pinned with `"*"`.

2.2 TypeScript project setup

A shared `tsconfig.base.json` enforces a strict suite: `strict`, `noUncheckedIndexedAccess`, `noImplicitOverride`, `noFallthroughCasesInSwitch`, `noImplicitReturns`, `exactOptionalPropertyTypes`, `isolatedModules`, and `verbatimModuleSyntax`, targeting ES2022 with `composite` incremental builds. The root `tsconfig.json` is a solution file referencing `types`, `engine`, `minigame-harness`, `ai`, and `web`. The `server` is intentionally *not* referenced — it builds independently under NodeNext module resolution because it is a Node application rather than a bundled library.

3 The Engine

The engine is pure TypeScript with no React, no DOM, no wall clock, and no `Math.random`. Its barrel (`packages/engine/src/index.ts`) re-exports the RNG, tunables, state, actions, events, board, `init`, and reducer.

3.1 Determinism

Randomness lives *inside* game state as a serializable `RngState`. The generator is a seeded mulberry32 (`rng.ts`); each draw returns both a value and the *next* state, which the caller stores back into `GameState`. Because the only source of entropy is `state.rng`, a game replays exactly from `seed` plus its ordered action log. The UI and server are free to use the wall clock (`performance.now()`, `Date.now()`, `Math.random` for timing jitter) because none of that crosses the engine boundary.

3.2 Game state

```
interface GameState {
  seed: number;
  rng: RngState;
  tunables: GameTunables;
  board: readonly Square[];
  players: readonly PlayerState[];
  activePlayerIndex: number; // index into players
  phase: TurnPhase;
  ownership: Record<number, PlayerId>; // squareId -> owner; missing = buyable
  lastRoll: readonly number[] | null;
  doublesCount: number; // 3 in a row -> jail; reset each turn
  round: number; // laps; roundCap force-ends the game
  pendingMinigame: MinigameRequest | null;
  winnerId: PlayerId | null;
}
```

A `PlayerState` carries `id`, `name`, `isAI`, `money`, `position`, `inJail`, `jailTurns`, `bankrupt`. A `Square` is `id`, `name`, `type` with optional `PropertyData` (`price`, `baseRent`, `group`).

3.3 Turn phases

The `TurnPhase` union is:

```
AWAITING_ROLL → MOVING → RESOLVING_SQUARE →
AWAITING_BUY_DECISION | RENT_SHOWDOWN → TURN_END → GAME_OVER
```

`MOVING` and `RESOLVING_SQUARE` are transient labels — the reducer threads straight through them and never rests there. The two phases the engine genuinely *parks* on are `RENT_SHOWDOWN` (waiting for a minigame result) and `GAME_OVER` (terminal). `AWAITING_BUY_DECISION` and `TURN_END` await a player action.

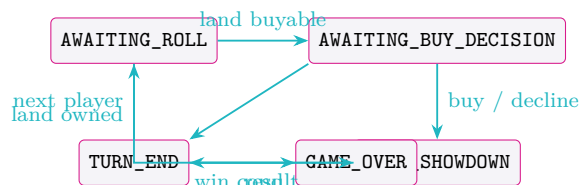


Figure 2: Phase transitions the engine actually rests on (transient `MOVING/RESOLVING_SQUARE` elided).

3.4 Actions and events

Actions are the only way to mutate state. The set is:

```
type GameAction =
  | { type: "ROLL_DICE" }
  | { type: "BUY_PROPERTY" }
  | { type: "DECLINE_BUY" }
  | { type: "PAY_JAIL_FINE" }
  | { type: "SUBMIT_MINIGAME_RESULT"; result: MinigameResult }
  | { type: "END_TURN" }
  | { type: "DECLARE_BANKRUPT"; playerId: PlayerId };
```

The reducer is a pure `reduce(state, action)` returning `{ state, events }`. Emitted events — `DICE_ROLLED`, `PLAYER_MOVED`, `PROPERTY_BOUGHT`, `MINIGAME_REQUESTED`, `RENT_PAID`,

SENT_TO_JAIL, PLAYER_BANKRUPT, TURN_ENDED, GAME_OVER — let clients react (animate, log telemetry, count showdowns) without re-deriving state.

3.5 Square resolution and the rent flow

Landing dispatches by square type. GO_TO_JAIL jails the player; tax deducts `taxAmount`; CHANCE and COMMUNITY are deliberately inert and resolve like Free Parking in the MVP. A property landed on:

- **unowned** → phase AWAITING_BUY_DECISION;
- **owned by the lander** → nothing;
- **owned by an opponent** → a Rent Showdown is opened.

Opening a showdown builds a `MinigameRequest` whose participants are ordered [**payer (index 0)**, **owner (index 1)**], parks the phase at RENT_SHOWDOWN, and emits MINIGAME_REQUESTED. This **payer-is-0, owner-is-1 ordering is load-bearing** and assumed everywhere downstream (rent multiplier resolution, server adjudication, the client's AI-seat detection).

3.6 Win conditions

On END_TURN the engine checks, in order:

1. **Last player standing.** If at most one solvent player remains, that player wins.
2. **Lap counting.** When the turn pointer wraps back to the first solvent player, `round` increments.
3. **Round cap.** If `roundCap > 0` and `round` exceeds it, the game force-ends and the *richest* player wins by the configured `tiebreakMetric`.

`netWorth` is cash plus the price of owned properties, unless the tiebreak metric is "CASH" (cash only). The round cap exists so a game always terminates — see the economy section for why that matters today.

4 The Minigame Harness

The harness is the seam that makes the minigame swappable across transports. It is renderer-agnostic — no DOM imports — so the same adjudication code runs in the browser and on the server.

4.1 Interface and registry

```
interface Minigame {
  id: MinigameId;
  minPlayers: number;
  maxPlayers: number;
  play(request: MinigameRequest): Promise<MinigameResult>;
}
```

A `Minigame` must always resolve — on failure it returns status `ABORTED` rather than rejecting. The `MinigameRegistry` maps ids to implementations (`register` throws on duplicates) and `run(request)` resolves a request end-to-end. This registry is where hotseat, AI, and online modes swap implementations in.

4.2 Reflex Tap Duel

The shared contract lives in `reflex-tap-duel.ts`; the actual renderer is in the web client. Configuration:

```
const DEFAULT_REFLEX_TAP_DUEL_CONFIG = {
  minDelayMs: 1500,      // earliest the "go" can appear
  maxDelayMs: 4000,      // latest the "go" can appear
  drawWindowMs: 25,      // reactions within this gap are a draw
  minHumanReactionMs: 100, // faster than this is treated as anticipation
};
```

4.3 Adjudication

`adjudicateReflexDuel(p0, p1, payerId, ownerId, drawWindowMs)` is pure and exhaustive over the edge cases (with `p0 = payer`, `p1 = owner`):

- both false-start → **ABORTED**;
- exactly one false-start → the other player wins;
- both reactions null (no tap) → **ABORTED**;
- exactly one null → the player who tapped wins;
- reaction gap $\leq$ `drawWindowMs` → **DRAW**;
- otherwise the faster reaction wins.

The result is a `MinigameResult` (`mingameId`, `status`, `outcome`, `ranking`, `metrics`) carrying `P0_WIN` | `P1_WIN` | `DRAW` and `COMPLETED` | `ABORTED`. The guiding principle of the whole seam: *the minigame decides who won; the engine decides what that win is worth.*

5 The AI

The AI package is pure and deterministic — it does not call the engine's RNG, and its only nondeterminism (duel reflexes) takes an injectable random source so tests can pin it.

5.1 Turn decisions

`decideAction(state, playerId)` returns the next `GameAction` or `null` when it is not this bot's turn (or the game is in a showdown or over). The policy is intentionally simple:

- **AWAITING_ROLL**: if jailed and the fine is affordable, pay it; otherwise roll.
- **AWAITING_BUY_DECISION**: buy when `price <= money * 0.6` (the `BUY_CASH_FRACTION` guard), else decline.
- **TURN_END**: end the turn.

Notably the AI never returns `SUBMIT_MINIGAME_RESULT` — the host drives the duel and feeds the result back.

5.2 Duel reflexes

`aiReflexInput(skill, rng)` produces a `ReflexInput` for a bot seat. False-start probability is $0.12 * (1 - \text{skill})$; reaction mean is $600 - 350 * \text{skill}$ milliseconds with up to ± 60 ms of jitter, floored at 250 ms. A `skill` of 0.6 is the default opponent, landing a believable human-ish reaction band.

6 The Online Server

The server wraps the engine in a Colyseus room and is fully **authoritative**: clients send intents and taps, never results. It listens on `PORT` (default 2567) and defines a single "game" room with `maxClients = 2`.

6.1 Room lifecycle

`GameRoom` holds the `GameState`, a seat map (`sessionId` $\rightarrow$ `PlayerId`), per-seat taps, and timers. The first two joiners take seats `p0` and `p1`; the game starts (seeded with `Date.now()`) once both are present. A consented leave ends the room; a network drop opens a 30-second reconnection window, after which the server re-sends state to the returning client.

6.2 Action handling and the legality gate

Incoming actions pass through `isLegalAction(state, playerId, action)` before reaching the reducer. Where the reducer silently no-ops an illegal action (fine for local play), the online path must *reject* it with an error message, so `validate.ts` mirrors the reducer's guards: no actions in `GAME_OVER`/`RENT_SHOWDOWN`, only on your turn, only when not bankrupt, and each action only in its proper phase. Legal actions are reduced and the new state is broadcast; if the phase becomes `RENT_SHOWDOWN`, the server starts a showdown.

6.3 Server-run showdown

1. **start** — taps are cleared and `showdown:start` is broadcast with the property's `baseRent`.
2. **go** — after a randomized delay (`minDelayMs` plus a random span up to `maxDelayMs`), `showdown:go` is broadcast and a 5-second tap timeout is armed.
3. **tap** — each seat may submit one tap; once both arrive (or the timeout fires, filling absentees with a `MISSING_TAP`) the showdown resolves.
4. **resolve** — `adjudicateShowdown` sanitizes both taps, adjudicates via the harness, and the result is fed back through the reducer as `SUBMIT_MINIGAME_RESULT`. New state is broadcast.

Anti-cheat: `sanitizeTap` demotes any non-false-start reaction faster than `minHumanReactionMs` (100 ms) to a false start, so a client cannot "win" by anticipating or spoofing an impossibly fast tap. Crucially, the client wire protocol's `ClientActionType` omits `SUBMIT_MINIGAME_RESULT` entirely — the result type a client could forge is simply not part of the messages it is allowed to send.

6.4 Wire protocol

Client-to-server (C2S) carries `action` and `tap` messages. Server-to-client (S2C) carries `state` (a generic `StateMessage` with the player's own id, kept generic so `types` need not depend on `engine`), `showdown:start`, `showdown:go`, and `error`. Tap timing is measured *client-side from receipt of the "go" signal*, not from any shared wall clock, which keeps latency from biasing the duel.

7 The Web Client

The client is React + Vite with Zustand for state. The store *mirrors* engine state rather than replacing it: the engine's `GameState` remains the source of truth, and the store pipes every dispatch through the real `reduce()`.

7.1 Modes

`App.tsx` is a mode switcher: `menu`, `hotseat`, `ai` (hotseat reused with a bot seat), `duel` (a board-less fairness practice), and `online`.

7.2 Components

- **Board** — 40 squares on an 11×11 CSS-grid ring, GO at bottom-right, counter-clockwise. Eight district colors, owner-inset borders, pawns, and a center panel showing round and phase.
- **Hud** — phase, round, last roll, and per-player cards (money, position, jail/bankrupt status).
- **ReflexTapDuel** — the hotseat (one-keyboard) duel: payer taps A, owner taps L; red arms to green after a randomized delay; a bot seat can be sampled up front. Measures with `performance.now()`.
- **OnlineReflexDuel** — the single-seat, server-driven duel: red on "start", green on "go", tap on Space/Enter, then wait for the authoritative result.
- **DuelSignal** — the shared red/green visual, deliberately colorblind-safe: shape, glyph, text, and aria-label all change, not just hue.
- **HotseatGame / OnlineGame** — the two game shells, dispatching through the reducer or through `sendAction` respectively.
- **DuelPractice** — a board-less "Phase 0 fairness gate" running duels back-to-back with a scoreboard (win rate, average reaction, false starts, draws).
- **DebugPanel** — a dev-only panel (hotseat/practice) to restart with a seed, teleport the active player, or set money via a local `debugPatch` that bypasses the reducer.

7.3 Stores and telemetry

`gameStore` holds the mirrored state, the AI seat/skill (p1 at skill 0.6), a showdown counter derived from `MINIGAME_REQUESTED` events, and a duel log. `onlineStore` tracks connection status (`idle/connecting/waiting/playing/reconnecting/error/left`) and a `showdown` signal whose `seq` field bumps on each phase change to disambiguate a stale "start" from a fresh "go". `net/onlineClient` is a thin, React-free wrapper over `colyseus.js` that stores the reconnection token and maps each server message to a handler. `telemetry/duel` flattens each result into a serializable `DuelRecord` and aggregates win rates and average reaction (over non-false-start taps only) — the data behind the fairness gate.

8 Economy and Tuning

8.1 Default tunables

```
const DEFAULT_TUNABLES = {
  startingMoney: 1200,
  passGoSalary: 75,
  taxAmount: 120,
  boardSize: 40,
  diceCount: 2,
  diceSides: 6,
  jail: { jailSquareId: 10, fine: 50, maxTurns: 3, releaseOnDoubles: true },
  rentMultipliers: { ownerWin: 1.5, payerWin: 0.5, draw: 1.0, aborted: 1.0 },
  rentFraction: 0.2, // baseRent = max(rentFloor, floor(price * 0.2))
  rentFloor: 8,
  rentEscalationStep: 0, // off by default; held for a playtest pass
  rentEscalationCap: 4,
  roundCap: 30,
  tiebreakMetric: "NET_WORTH",
```

```
};
```

8.2 How rent is priced

A property's `baseRent` is computed *once at board build* as `max(rentFloor, floor(price * rentFraction))` — 20% of price, floored at 8. At showdown time the duel outcome selects a multiplier (owner win $\times 1.5$, payer win $\times 0.5$, draw $\times 1.0$, aborted $\times 1.0$) and the payment is `round(baseRent * multiplier * escalation)`. If the payer cannot cover it, they pay all their cash and go bankrupt to the owner.

The `escalation` factor stands in for the missing houses and hotels: rent climbs with how many properties the owner holds, as `min(rentEscalationCap, 1 + rentEscalationStep * (owned - 1))` — a lone property never escalates, and the factor is capped. With `rentEscalationStep = 0` (the default) the factor is always 1, so rent is unescalated until a playtest decides the value.

8.3 The convergence problem

The MVP cut houses and hotels. In real Monopoly those are what scale rent steeply enough to bankrupt players; without them, flat rent plus the modest $0.5\times-1.5\times$ duel multiplier produces rents that stay small relative to recurring income. A bot-vs-bot simulation under the original tunables (`startingMoney 1500`, `passGoSalary 200`, flat low rents) showed **9 of 10 games never reaching GAME_OVER** — every player's net worth only grew, so nobody went bankrupt and the last-player-standing condition could never fire. The cause was confirmed to be tunables, not a bug: a punitive re-run (low salary, high tax, low starting money) finished 10/10 games.

8.4 The interim fix (shipped)

The current defaults above are the response: lower starting money and pass-GO salary, rent raised to 20% of price (now driven by the `rentFraction` / `rentFloor` tunables), and a `roundCap` of 30 with a `tiebreakMetric` of `NET_WORTH` that force-ends a stalled game to the richest player. Under these, the simulation finishes **30 of 30 games, median ≈ 62 turns**.

8.5 Measuring convergence — the sim harness

Balance is now measured by a committed, headless bot-vs-bot harness (`packages/sim`), so the convergence question is data rather than anecdote. It drives the real engine with the real AI and seeded reflex duels — fully deterministic from a seed — and classifies each game's ending as *elimination* (a last-player-standing win), *cap* (the round cap force-ended it on net worth), or *timeout* (never resolved within the step guard — a balance smell). Run it with `npm run sim` (e.g. `--games 300 --players 4 --escalation 0.5`).

The harness reproduces the historical 2-player baseline exactly (26.7% elimination), which validates it against the earlier ad-hoc runs. It also reveals that player count matters more than previously recorded: with the shipped defaults, a 2-player game eliminates 26.7% of the time, but a 4-player game almost never does (0.7%) — spreading wealth across four players keeps flat rent from bankrupting anyone, so nearly every 4-player game ends on the cap.

8.6 Open question — elimination vs. points

Rent escalation is now *implemented* (the `rentEscalationStep` / `rentEscalationCap` tunables above) but ships **off**, so the default balance is unchanged and games still mostly end on the cap. The data shows the lever works: **step 0.5, cap 4** lifts 4-player elimination from 0.7% to 53%, and a steeper **step 1.0, cap 5** reaches 80%. What remains is a *feel* decision — how often a game should end by knocking someone out versus on points, and at what player count — which

Players	Escalation	Elimination	Cap	Median turns
2	off (step 0)	26.7%	73.3%	54
2	step 0.5, cap 4	84.0%	16.0%	37
4	off (step 0)	0.7%	99.3%	87
4	step 0.5, cap 4	53.0%	47.0%	72
4	step 1.0, cap 5	80.3%	19.7%	58

Table 2: Ending mix over 300 seeds per row (skill 0.6). All rows finish 100% of games; escalation moves wins from the cap toward bankruptcy elimination.

is a playtest call, not a code one. QA has flagged a prerequisite: the reflex-duel’s *fairness* should be validated with a short human, duel-only session (the Phase 0 fairness gate / `DuelPractice` mode) *before* committing an escalation value — there is no point tuning the stakes of a duel that does not yet feel fair.

9 Building, Testing, Running

9.1 Scripts

From the repository root:

```
pnpm install
pnpm dev          // apps/web dev server
pnpm build        // apps/web production build
pnpm typecheck    // tsc across workspaces (--if-present)
pnpm test         // vitest across workspaces (--if-present)
pnpm sim          // headless balance simulation (see Economy)
pnpm lint         // eslint .
```

The server and the sim run independently with `tsx`; neither is a bundled library, so both use NodeNext resolution and are left out of the root TypeScript solution’s project references.

9.2 Tests

Vitest suites cover the load-bearing pure logic: engine determinism and game logic, AI decisions and reflexes, reflex adjudication, and the server’s showdown sanitization and action legality. The web client has no automated tests yet — its behavior is exercised through the fairness-gate playtest mode instead.

Package	Test files
engine	<code>determinism.test.ts</code> , <code>game-logic.test.ts</code>
ai	<code>decide.test.ts</code> , <code>reflex.test.ts</code>
minigame-harness	<code>reflex-adjudicate.test.ts</code>
server	<code>showdown.test.ts</code> , <code>validate.test.ts</code>
sim	<code>simulate.test.ts</code> (determinism, termination, escalation)

Table 3: Current automated test coverage.

9.3 Reproducing a game

Because the engine is deterministic, a bug report needs only the `seed` and the action log. Replaying those through `reduce()` reconstructs the exact state at any step — the same property the bot-vs-bot economy simulations rely on, and the reason the determinism boundary is guarded so strictly.

10 Glossary

Rent Showdown

The reflex duel triggered by landing on an opponent-owned property; its outcome scales the rent.

Tunables

The data-driven economy parameters (`DEFAULT_TUNABLES`) that configure money, rent, jail, and win conditions without code changes.

Round cap

A lap limit (default 30) that force-ends a stalled game to the richest player, guaranteeing termination.

Participant ordering

The load-bearing convention that minigame participant index 0 is the payer/lander and index 1 is the owner.

Determinism boundary

The rule that only the engine's seeded RNG may affect game state; the UI and server may use the wall clock freely.

Authoritative server

The model where the server alone adjudicates duels and submits results; clients send only intents and taps.